



POLITECNICO DI BARI

DEPARTMENT OF ELECTRICAL AND INFORMATION ENGINEERING
Master Degree in Automation Engineering

Mobile Robotics

Autonomous Leader–Follower Navigation with Visual Target Recovery

Supervisors

Prof. Eng. Luca De Cicco
Eng. Nunzio Barone

Student

Luigi Pio Ficco

Academic Year 2025 – 2026

Abstract

This documentation presents the design, implementation and experimental evaluation of a vision-based leader-follower navigation system for a TurtleBot3 robot. The project was developed entirely in simulation, using ROS2 Jazzy as the robotics framework and Gazebo Harmonic as the simulation environment.

The objective is to enable a follower robot to track and pursue a moving target, identified by an ArUco marker mounted on its rear panel, keeping a prescribed following distance while avoiding obstacles. The follower has no prior knowledge of the target's route and relies exclusively on onboard perception: an RGB camera for detection and a 2D LiDAR for safety.

The distinguishing contribution of this work is not the pursuit itself, which is a classical problem, but the *recovery of the target after a loss of visibility*. When the marker leaves the camera field of view, the system estimates the target's motion from the last available observations and continues the pursuit blind for a bounded time horizon; if that fails, it falls back on an active search, rotating in place until the marker reappears.

The system deliberately uses *no map, no SLAM and no global planner*: navigation is entirely reactive and perception-driven. The design is validated quantitatively by comparing the proposed predictive strategy against a purely reactive baseline, which stops as soon as the target is lost, over repeated runs on identical, deterministic target trajectories.

Contents

Abstract	1
1 Introduction and Objectives	4
1.1 Problem statement	4
1.2 Design Philosophy	4
2 System Overview	5
2.1 Architecture	5
2.2 Topic interface	5
2.3 Software environment	5
3 Simulation Environment	6
3.1 The arena	6
3.2 The two robots	6
3.2.1 The camera was replaced	6
3.2.2 How the marker is mounted on the target	6
4 Perception: Marker Detection and Pose Estimation	7
4.1 The marker	7
4.2 From the image to the target position	7
4.3 The detector node	8
4.4 The dominant loss mechanism: incidence angle	8
5 Reference Frames	8
5.1 The velocity must be estimated in a fixed frame	8
5.2 Two things kept out of the transform tree on purpose	9
6 Pursuit Control	9
6.1 The control law	9
6.2 What each term buys	10
6.3 Gain tuning	11
7 Motion Prediction	11
7.1 Estimating how fast the target is moving	11
7.2 Predicting where it went	11
7.3 The same code serves both pursuit and prediction	12
7.4 The prediction horizon	12
8 The Recovery State Machine	12
8.1 States and transitions	12
8.2 Abandoning the prediction on a sensor contradiction	13
8.3 The two strategies under comparison	13
9 Safety Arbitration	14
9.1 The conflict	14
9.2 How the LiDAR limits the visual command	14
10 Experimental Infrastructure	16
10.1 The target as a reproducible stimulus	16
10.2 Ground truth for the target, and why it is legitimate	16



11 Testing and Validation	16
11.1 Experimental protocol	16
11.2 Metrics	16
11.3 Results	17
11.3.1 What the comparison shows	17
11.3.2 Why the benefit appears where it does	18
11.4 Summary of the work	18
11.5 Limitations	18
11.6 Future work	19
11.7 Repository	19

1 Introduction and Objectives

1.1 Problem statement

Leader–follower navigation is the problem of making one mobile robot (the *follower*) maintain a prescribed spatial relationship with respect to another moving agent (the *leader* or *target*), using only onboard sensing. It is the abstraction behind a wide range of applications: automated load carriers that follow a warehouse operator, convoys of ground vehicles in which only the first is driven, service robots that accompany a person through a building.

The hard part is not the pursuit itself, which is a classical control problem, but what happens when the target disappears. A camera has a finite field of view; a marker is readable only within a limited range and incidence angle. The target *will* vanish, repeatedly, and what the follower does in those seconds is what separates a demonstration from a usable system.

This is the behaviour the project proposal commits to:

[...] When the target leaves the camera field of view, the system estimates its most likely position from the last available observations and continues the pursuit for a limited time before switching to an active search behaviour. The final system will be evaluated by comparing a purely reactive following strategy, which stops as soon as the target is lost, against the proposed predictive strategy.

Three commitments follow from this statement and structure the whole document:

- **Perception-only operation.** The follower does not know the target’s route, and no map of the environment is used. Every decision is taken from the current camera and LiDAR readings plus a short memory of past detections.
- **Bounded blind pursuit.** Prediction is explicitly limited in time. An unbounded extrapolation would eventually chase a fiction; the horizon is a design parameter with a measurable optimum, studied in Section 7.4.
- **Quantitative comparison.** The system must be evaluated, not merely demonstrated. Two strategies, identical in every respect except the predictive stage, are run on identical deterministic trajectories, and the difference is measured.

1.2 Design Philosophy

This project deliberately uses none of the standard ROS2 autonomy stack: `slam_toolbox` for mapping, Nav2 for global planning and control, or `robot_localization` for state estimation. The omission is a design decision rather than a simplification.

The reason is that a map would not help. A global planner answers the question “how do I get from here to a known goal pose while avoiding known obstacles?”. The follower does not have that question. Its goal is not a pose in a map: it is a moving object whose position is only known through the camera, from instant to instant. What the follower needs is:

- the *relative* position of the target: provided by the camera;
- knowledge of what is immediately in front of it: provided by the LiDAR;
- a short-term memory of where the target was: provided by odometry integrated over a few seconds.

An occupancy grid adds no information to any of these three. It would add a substantial software stack to configure, tune and debug, in exchange for nothing. The consequence is stated honestly in Section 11.5: without a map the follower cannot reason about corridors, rooms or exits, so once it has lost the target it can only rotate in place and wait for it to reappear. That is a structural ceiling on the search, and it is the price of the choice.

2 System Overview

2.1 Architecture

Five processes, each with one responsibility: the **detector** recognises the marker and publishes where the target is; the **controller** decides how to move; the **driver** moves the target along fixed waypoints; the **logger** observes and records without ever intervening. Gazebo and the **ros_gz** bridge supply the sensor data.

The division into five processes is not arbitrary. Because the nodes talk only through topics, the detector could be replaced by a learned one without a single line of the controller changing: the interface between them is a pose on a topic, not a function call. The same property makes the system easy to inspect: every link in the graph can be read from outside with **ros2 topic echo**, which is how most of the debugging in this project was actually done, and which a variable inside a single monolithic node would not allow. The cost is the latency of passing messages between processes, negligible at these rates.

2.2 Topic interface

Topic	Type	Meaning
/camera/image_raw	Image	follower camera, 10 Hz
/scan	LaserScan	LiDAR, 360°
/target/pose	PoseStamped	target position in odom, published only when the marker is seen
/cmd_vel	TwistStamped	velocity command to the follower
/follower/state	String	current state of the machine

Loss of visibility is signalled by silence. The detector publishes nothing when it cannot see the marker, and the controller infers the loss from a timeout rather than from an explicit flag. One message type suffices and the semantics are unambiguous: every message on **/target/pose** is by construction a successful detection. A flag could also remain latched at its last value if the detector died; a timeout cannot.

2.3 Software environment

Table 1: Software stack.

Component	Version / notes
Framework	ROS2 Jazzy
Simulator	Gazebo Harmonic, via ros_gz_sim / ros_gz_bridge
Vision	OpenCV, cv2.aruco , dictionary DICT_4X4_50
Robot platform	TurtleBot3 Burger (official ROBOTIS models)
Containerisation	Docker, image based on ros:jazzy
Language	Python 3 (rclpy , NumPy, OpenCV)
Visualisation	RViz2

The entire workspace is containerised. The image installs ROS2, the Gazebo bridge, **cv_bridge**, **tf2_geometry_msgs**, OpenCV and the Python scientific stack. The workspace is mounted as a volume rather than copied into the image, so that source changes require only a **colcon build** and not an image rebuild.

3 Simulation Environment

3.1 The arena

The environment is a rectangle of 10×8 m enclosed by walls, with a chicane formed by two vertical blades: one hanging from the top edge, the other rising from the bottom. The free gaps therefore alternate, and any route crossing the arena is forced to weave. Three fixed ArUco panels are mounted flush against the walls.

It was built for this project rather than taken from the official package, because the official arenas are dimensioned for *one* robot navigating alone. A leader–follower pair separated by 1.5 m occupies a band of nearly three metres along the route, so the size of the environment follows from the following distance rather than from the size of a single robot.

The arena is generated by a short script, `generate_world.py`, from a geometry written once in `worlds.py`. Drawing it directly in Gazebo would have worked just as well for the simulation, but the same geometry is needed a second time, to check that the target’s routes keep a safe distance from the walls, and describing it in two places is how the two come to disagree. That is not hypothetical: it happened, the world was modified and the check was not, and a robot ended up spawning inside an obstacle with no error message at all. Generating the world from the geometry leaves a single source of truth.

3.2 The two robots

Both derive from the official ROBOTIS models and differ in one respect each. The follower carries a replaced camera, for the reason measured in Section 3.2.1. The target carries the ArUco panel.

3.2.1 The camera was replaced

The follower does not carry the camera ROBOTIS ships. The switch was driven by simulation performance: the official `turtlebot3_burger_cam` runs a fisheye lens at 320×240 pixels and 30 Hz, and rendering a fisheye view in Gazebo is markedly more expensive than a standard camera at comparable resolution, since the wide field of view is typically built from several internal render passes rather than one. With it in the scene, the simulation was too slow to work with comfortably.

The replacement is a pinhole camera at 480×360 , running at 10 Hz: despite the higher resolution, the lower frame rate alone cuts the raw pixel throughput by a quarter, and the simpler rendering path removes the extra cost of the fisheye projection on top of that.

The choice also happens to be favourable for marker detection, although this was not the motivation and was not tested against the original fisheye directly. Wide-angle lenses are a documented source of difficulty for standard ArUco detection: distortion and the non-uniform resolution they introduce degrade corner localisation badly enough that the methods used for a perspective camera do not carry over directly to a fisheye one, to the point that dedicated variants of the detector have been proposed for panoramic cameras. `solvePnP` itself assumes a pinhole projection, which a fisheye violates outright, so any pose it returned on fisheye input would be distorted towards the edges of the image regardless of resolution.

With the pinhole camera actually deployed, the resulting detection range is roughly 1.8 m. That figure governs the difficulty of the whole scenario: it is what makes a following distance of 1.5 m a demanding choice rather than a comfortable one.

3.2.2 How the marker is mounted on the target

The panel is a textured square of 0.18 m placed 0.108 m behind the centre of the robot and 0.15 m above the ground, with a yaw that turns its textured face backwards, towards the pursuing follower.

```
1 <visual name="aruco_plate">
2   <pose>-0.108 0 0.150 0 0 1.5708</pose>
3   <cast_shadows>>false</cast_shadows>
4   <geometry><box><size>0.18 0.008 0.18</size></box></geometry>
5   <material>
6     <diffuse>1 1 1</diffuse><ambient>1 1 1</ambient>
7     <pbr><metal>
8       <albedo_map>materials/textures/target_marker.png</albedo_map>
9       <roughness>1.0</roughness><metalness>0.0</metalness>
10    </metal></pbr>
11  </material>
12</visual>
```

Code 1: The whole modification to the target model.

It is a `visual` element of `base_link`, not a separate link: the marker is rigidly fixed to the robot, so it needs neither a joint nor a mass of its own. A separate link would only be justified if it moved relative to the body. A matching `collision` is declared alongside it, because the panel protrudes past the chassis and without it the follower's LiDAR would not see it.

The marker is on the rear, and that is the whole problem. Mounting it at the back is the natural geometry of pursuit, the follower is behind. But it is also the origin of the dominant loss mechanism of the entire project, described in Section 4.4: the panel is readable only from roughly behind, so it becomes unreadable as soon as the target turns, well before any obstacle interposes.

Mounting several panels, one per side, would reduce the losses considerably, but at the cost of the very phenomenon this project sets out to study.

4 Perception: Marker Detection and Pose Estimation

4.1 The marker

An ArUco marker is a binary square: a thick black border around a 4×4 grid of cells encoding an identifier. The dictionary used, `DICT_4X4_50`, is not the set of all 2^{16} patterns: OpenCV generates it, like every predefined ArUco dictionary, by maximising the minimum Hamming distance between any two of its fifty elements *across all four rotations*, a property that matters because the detector does not know in advance how the marker is oriented in the image.

The practical consequence matters: a few misread cells do not produce a wrong identifier, they produce *no* identifier. That is the desirable way to fail.

The target carries ID 3; the three fixed panels on the walls carry 0, 1 and 2.

4.2 From the image to the target position

Finding the marker in the image is only half the job: the detector returns the pixel coordinates of its four corners, and what the controller needs is *how far away* and *in which direction* the target is.

Going from one to the other is the *Perspective-n-Point* problem: given four points whose positions are known on a square of known size, and their projections in the image, find the rotation and translation that produce them. Intuitively, a marker that appears small is far away, and one that appears as a trapezoid rather than a square is seen at an angle: PnP turns that intuition into numbers.

This is not implemented here. OpenCV solves it in `solvePnP`, and for a planar square it uses the closed-form `IPPE_SQUARE` method. The camera's intrinsic matrix, which the solver needs, arrives ready-made on the `/camera/camera_info` topic: in simulation it is exact, which is why no calibration step is required.

What the project does is supply the four *object points*, the corners of the black square in the marker's own frame, and convert the result into the two quantities the control law uses:

- the **distance** d , the length of the translation vector;
- the **bearing** β , the horizontal angle between the camera axis and the target, negative to the right.

The size passed to the solver is the black square. `marker_size` is the side of the black square, 0.15 m, not of the panel that carries it, which is 0.18. The white border is the quiet zone the detector needs to separate the marker from the background, and it is not part of the object the solver reasons about.

Passing the wrong one produces no error message: every distance comes out systematically too large, and the follower settles at the wrong distance in silence.

4.3 The detector node

The node subscribes to the camera stream, looks for the marker carrying the expected identifier, estimates its pose and publishes it in the fixed `odom` frame. Two interface decisions are worth stating.

It publishes only when it sees. No message is emitted when the marker is absent, so the controller infers the loss from the *silence* of the topic through a timeout. One message type suffices and the semantics are unambiguous: every message on `/target/pose` is by construction a successful detection. A flag could also stay latched at its last value if the detector died; a timeout cannot.

It filters by identifier. Four markers of the same dictionary exist in the arena, the three fixed panels and the target, and more than one is often visible in a single frame. A single line selects the one carrying ID 3, and the main trajectory deliberately passes alongside all three fixed panels so that this can be seen working.

4.4 The dominant loss mechanism: incidence angle

An experimental observation that shaped the entire scenario design deserves to be stated explicitly, because it is counter-intuitive.

The marker is mounted on the *rear* of the target, so it is readable only while the follower is looking at it more or less from behind. As soon as the target turns, the panel rotates away from the line of sight and the projected square degenerates until the decoder can no longer read it, and this happens well before any obstacle interposes itself.

The consequence is that in this system **the dominant cause of target loss is the geometry of the turn, not occlusion by obstacles**. This was verified by simulation: the frequency of loss correlates with the density of turns along the trajectory and is almost insensitive to the number of obstacles placed in the arena. The scenario design quantifies the conditions under which geometric occlusion *does* contribute.

5 Reference Frames

The same physical quantity, the position of the target, is handled in three frames, for three different reasons: the *optical* frame, where `solvePnP` produces it; `base_footprint`, where the control law needs distance and bearing relative to where the robot is pointing now; and `odom`, a fixed frame.

5.1 The velocity must be estimated in a fixed frame

The third one is the non-obvious part, and it is why the detector transforms the pose *before* publishing it rather than leaving it to the controller.

Estimating velocity by differentiating the position observed in the camera frame would differentiate a quantity that also moves because *the follower moves*. The result is the velocity of the target *relative to the follower*, not its velocity. And since the follower is turning precisely in the moments before a loss, extrapolating that estimate would predict a position systematically displaced in the wrong direction. Transforming into `odom` removes both spurious terms.

5.2 Two things kept out of the transform tree on purpose

TF2 organises reference frames as a tree, and any node can ask it to convert a point from one frame to another. Two things are deliberately kept out of that tree, and both omissions protect the validity of the experiment.

The true pose from the simulator. Gazebo knows exactly where both robots are. That information is published on a topic, but it is *not* inserted into `/tf`. If it were, a lookup somewhere in the controller could eventually resolve through it, and the follower would end up using knowledge it is not supposed to have: the results would silently become those of a robot that cheats. Keeping it off the tree makes that impossible by construction rather than by discipline.

The target's own frames. The target robot has its own DiffDrive plugin, which produces frames named `odom` and `base_footprint`: the same names the follower uses. Publishing both on `/tf` would create two robots claiming the same names in the same tree, and every lookup would resolve non-deterministically: sometimes it would find the follower, sometimes the target. The target's tree is therefore routed to `/target/tf`, where nothing listens to it.

There is also no `map` frame, for the straightforward reason that there is no SLAM.

6 Pursuit Control

6.1 The control law

The TurtleBot3 Burger is a differential-drive platform. It has exactly two control inputs, a linear velocity v and an angular velocity ω , and it cannot translate sideways: to move to its left it must first turn. This is the non-holonomic constraint, and it shapes the whole control design.

The task is described by two errors:

d	distance to the target	measured by the camera
β	bearing to the target	measured by the camera, positive to the left
d^*	desired following distance	1.5 m, a parameter
$e_d = d - d^*$	distance error	positive when too far
$e_\beta = \beta$	bearing error	zero when the target is straight ahead

The law adopted is a pair of proportional loops, one per input:

$$v = (\hat{v}_t \cdot \hat{u}_{los} + K_v e_d) \cos(\min(|e_\beta|, \pi/2)), \quad \omega = K_\omega e_\beta. \quad (1)$$

$K_v = 0.6$	gain of the distance loop: how hard the follower accelerates per metre of error
$K_\omega = 1.5$	gain of the bearing loop: how hard it turns per radian of error
$\hat{v}_t$	estimated velocity of the target, a vector, obtained from the regression of Section 7.1
$\hat{u}_{los}$	unit vector along the line of sight, from the follower to the target

Where the law comes from. The two proportional terms are the textbook starting point: drive each error to zero with a command proportional to it. That much is standard, and it is what a first implementation would look like.

The other two terms were each added to fix a specific, observed defect, and they are the part worth defending.

The product $\hat{v}_t \cdot \hat{u}_{los}$ is a scalar, the component of the target's velocity along the direction the follower is looking. Only that component changes the distance; the perpendicular one changes the bearing instead, and is handled by the other loop. It is a feed-forward term: the follower moves at the target's speed because it knows the target's speed, not because an error tells it to.

The $\cos(e_\beta)$ factor is the only coupling between the two loops, and it encodes the non-holonomic constraint: if the target is 90° to one side, moving forward does not reduce the distance at all, the robot would describe an arc and get further away first. The cosine suppresses translation while the robot is badly aligned, so it turns first and drives second. Saturating at $\pi/2$ keeps it from going negative, which would command reverse as a side effect of misalignment.

6.2 What each term buys

Two decoupled loops. For a unicycle the two errors act on channels that are close to independent near the operating point: bearing error is corrected by rotating, distance error by translating.

The cosine factor. It is the only coupling between them, and it encodes the non-holonomic constraint: with the target 90° to the side, advancing does not reduce the distance, the robot would describe an arc and initially move away. Suppressing translation while misaligned means rotating first and translating after. Saturating at $\pi/2$ keeps the cosine from going negative, which would command reverse motion as a side effect of a large bearing error.

Dead bands. 0.10 m on distance and 0.05 rad on bearing, chosen just above the measurement noise. Without them the noise on d produces a continuous alternation of small forward and backward commands: a limit cycle that shows up as jitter in simulation and would wear the actuators on hardware.

Feed-forward on the target velocity. This is the term that repays explanation, because its absence is not obvious.

A proportional loop tracking a moving reference has a lag. To hold station behind a target moving at v_t the follower must itself move at v_t , and with a purely proportional law that command can only come from a non-zero error:

$$e_{ss} = \frac{v_t}{K_v} = \frac{0.21}{0.60} = 0.35 \text{ m.} \quad (2)$$

The follower would settle at 1.85 m instead of 1.50, *beyond* the marker's 1.8 m detection range: without the feed-forward term the target would be lost simply by holding station on a straight line, before it ever turned a corner.

An integral term would remove it, at the price of anti-windup logic against a saturating actuator. But the target velocity is *already estimated* here for the prediction, so it can simply be fed forward, and the proportional term is left to correct only the residual. An integrator compensates a disturbance that cannot be measured; here it can.

Backing off is allowed only while the target is getting closer. The control law is a distance regulator, so if the follower ends up nearer than the setpoint the commanded velocity comes out negative and the robot reverses. That happens regularly, because the target slows down considerably in turns while the follower is still closing on it.

Reversing, however, is only sensible if the gap is still shrinking. If the target has already started moving away, the gap is closing by itself, and backing off would open it at the sum of both speeds: pushing the marker out of detection range faster than simply holding position would. The follower therefore reverses only when the measured distance is actually decreasing; otherwise it stops and turns instead, which is the right thing to do in the moment right after regaining contact.

The setpoint ramps after a re-acquisition. Regaining contact at close range leaves a large negative error, and the follower would sit still until the target receded past d^* : roughly seven seconds, during which a single detection gap restarts the cycle. The setpoint therefore starts at the distance at which contact was regained and rises to nominal over five seconds. Right after finding the target again, the priority is not to lose it, not to restore the geometry.

6.3 Gain tuning

The gains were tuned following the standard sequence for cascaded loops: first the angular loop in isolation, then the linear loop, then the interaction.

Table 2: Control parameters and their rationale.

Parameter	Value	Rationale
K_ω	1.50	Fast enough to keep the target centred during turns. Higher values produce visible oscillation about the bearing setpoint, because the camera update period (0.1 s) introduces a delay in the loop.
K_v	0.60	With e_d of order 0.5 m the command saturates, which is desirable: recovery of distance should be as fast as the platform allows.
d^*	1.50 m	See the scenario design: this is the parameter that governs scenario difficulty.
Dead bands	0.10 m, 0.05 rad	Above the measurement noise floor.

7 Motion Prediction

When the marker disappears, the controller needs an answer to a simple question: *where is the target now?* It cannot see it, but it saw it a moment ago, and it can work out how fast it was going. This section describes how that estimate is produced and for how long it is trusted.

7.1 Estimating how fast the target is moving

The detector supplies a position roughly ten times a second. The controller keeps the last eight of them, a window of about 0.8 s, and works out the target's velocity by fitting a straight line through those eight points. The slope of that line is the velocity.

The obvious alternative is to take the last two positions, subtract them and divide by the time between them. That is what velocity means, and it would work if the positions were exact. They are not: each one carries a little measurement noise, and subtracting two noisy numbers and dividing by a small interval makes that noise larger, not smaller. Drawing a line through eight points instead lets the errors cancel against each other.

Two safeguards are applied on top. If fewer than five positions are available, which is the case immediately after regaining contact, when the history has just been cleared, no extrapolation is attempted at all and the follower assumes the target is where it last saw it. And the estimated speed is capped at what a TurtleBot3 can physically do, since the target is one.

7.2 Predicting where it went

Once the speed is known, the prediction itself is deliberately plain: the target is assumed to carry on in a straight line at the speed it had. Its predicted position is the last one seen, plus that velocity multiplied by the time elapsed since.

A more elaborate model would need an extra quantity estimated from the same short, noisy window, acceleration, or rate of turn, and those are harder to measure reliably than speed. The rate of turn in particular becomes unmeasurable at the exact moment the target goes out of view, which is when it would be needed.

One safeguard is worth mentioning. On a target that is standing still, the residual noise still produces a small apparent velocity. Without a minimum speed below which the estimate is treated as zero, the follower would extrapolate that noise and drift away from a target that has not moved at all.

7.3 The same code serves both pursuit and prediction

The FOLLOW and PREDICT states share the same code path: the target's position is *always* obtained by extrapolation. While the marker is visible the elapsed interval is a few hundredths of a second and the correction is negligible; when it disappears the interval grows and the extrapolation becomes the only information available.

The benefit is practical rather than aesthetic. The prediction runs on every control cycle, not only in the rare moments when the target vanishes, so a defect in it shows up immediately during ordinary pursuit instead of hiding until the one situation where it matters most.

7.4 The prediction horizon

The horizon T_h is how long the follower keeps trusting the estimate after the marker disappears. It embodies a trade-off: too short and the robot gives up immediately, too long and it chases a wrong estimate away from the real target.

The value adopted is $T_h = 3$ s, and it follows from two quantities that are already fixed.

How long the prediction needs to last. The dominant loss is the incidence angle: the target turns, the panel goes edge-on, and it comes back into view once the turn is complete. A 90° turn at the target's angular speed takes roughly a second and a half. The horizon must comfortably cover that, or the prediction would be abandoned just before it pays off.

How long it can afford to last. The target travels at 0.21 m/s, so every second of blind pursuit is 0.21 m of ground covered on an estimate that assumes a straight line: while the target is, by hypothesis, turning. Past a few seconds the assumption is plainly false and the follower would be driving towards a point the target never reached.

Three seconds sits between the two: twice the duration of a turn, and short enough that a straight-line assumption about a turning target has not yet become absurd.

The horizon is not the only way out. Fixing a timeout would be a weak design on its own, because three seconds is right for a 90° turn and wrong for something else.

The prediction is therefore abandoned on a second criterion as well, described in Section 8.2: if the LiDAR reports a wall where the camera's estimate says the target should be, the estimate is unreachable and the timeout is not waited out. That criterion is not a tuned number, it is a contradiction between two sensors, and it holds regardless of the geometry.

8 The Recovery State Machine

8.1 States and transitions

The behaviour of the follower is governed by a finite state machine with three states.

FOLLOW

Nominal pursuit. The marker is visible and the control law of Section 6 is active.

PREDICT

The marker has been lost for more than `lost_timeout` (0.4 s). The follower continues the pursuit using the extrapolated position, at 70% of the nominal linear velocity. Moving more slowly while blind is deliberate, for two reasons. The estimate degrades with time, so the cost of an error grows with the distance travelled on it: at full speed, a position that is increasingly likely to be wrong would turn a small estimation error into a large positional one. It also leaves more margin for the safety arbitration (Section 9) to react if the estimated position happens to lie beyond an obstacle. The exact figure, 70%, was chosen empirically within that range rather than derived from a formula.

SEARCH

The follower rotates in place, scanning the surroundings, until the marker reappears.

Loss of visibility is detected by timeout rather than by an explicit flag, consistently with the interface choice of Section 4. The threshold of 0.4 s corresponds to four missed camera frames at

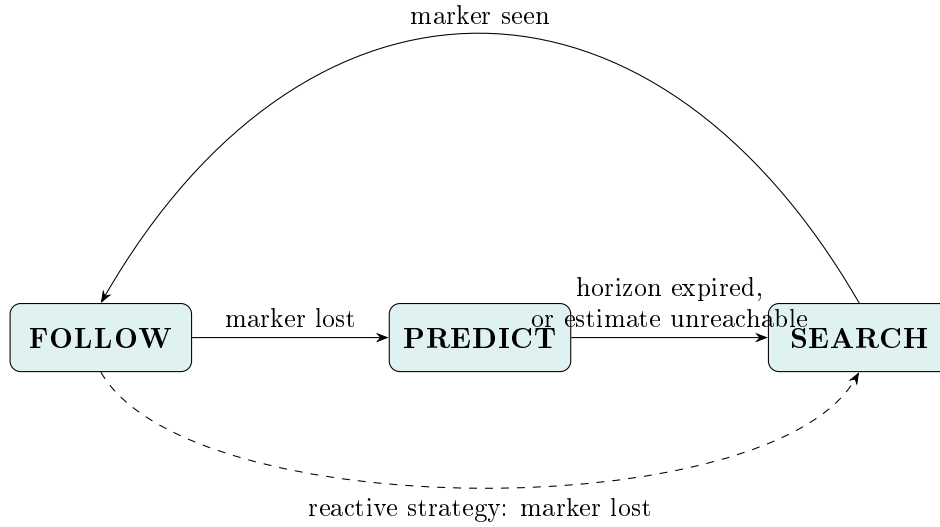


Figure 1: Recovery state machine. In the reactive strategy the PREDICT stage is skipped: a loss leads straight to SEARCH, which is the behaviour the proposal prescribes for the baseline.

10 Hz: long enough not to trigger on an isolated failed detection, short enough that the reaction is prompt.

8.2 Abandoning the prediction on a sensor contradiction

The prediction is abandoned on two criteria, not one. The first is the horizon. The second is geometric: if the LiDAR reports an obstacle within braking distance in the direction of the estimated position, and the estimate lies well beyond it, the follower is *pinned* and the estimate cannot be reached.

This second criterion is more interesting than the first because it is not arbitrary. It is a contradiction between two sensors: the camera says the target is over there, the laser says there is a wall in between, and one of the two is extrapolating while the other is measuring.

Both conditions are required, and that is the point. A rule that aborted the prediction whenever *any* obstacle lay in that direction would not work here. In the chicane the target is lost precisely because the panel goes edge-on behind one of the blades, so an obstacle in that direction is the normal case, not the exception: with only that condition, almost every prediction would be aborted within a fraction of a second of the loss, and the predictive strategy would stop differing from the reactive one. The comparison would then measure two identical behaviours without anyone noticing.

Requiring both conditions, obstacle within braking distance *and* estimate well beyond it, avoids that. While there is room to keep closing, the pursuit continues and the safety layer handles the approach.

8.3 The two strategies under comparison

The whole experiment rests on comparing two behaviours, and they must differ in *one thing only*. Here is what each does at the moment the marker is lost:

predictive	extrapolates where the target is going and keeps driving towards it for three seconds; only if that fails does it stop and search
reactive	stops immediately and rotates in place until the marker reappears

In the code that is a single boolean:

```
1 def on_target_lost(self):
2     # The ONLY point where the two strategies diverge.
3     #
4     # The baseline stops and looks around, as the proposal prescribes:
5     if not self.use_pred:
6         self.enter_search('target lost -> SEARCH (reactive baseline)')
7     elif len(self.history) >= 3:
8         self.lost_since = self.now()
9         self.set_state(PREDICT, 'target lost -> PREDICT')
10    else:
11        self.enter_search('target lost -> SEARCH (too few samples)')
```

Code 2: The only difference between the two strategies.

With `use_prediction: false` the PREDICT state is skipped and a loss leads directly to the search. Everything else is identical: the same control law, the same gains, the same safety arbitration, the same search behaviour. The single independent variable is the blind pursuit stage, and that is what makes the comparison a controlled experiment rather than a demonstration.

That the baseline *stops* is the definition given in the proposal, not an implementation shortcut: “a purely reactive following strategy, which stops as soon as the target is lost”.

9 Safety Arbitration

9.1 The conflict

The project proposal identifies this problem directly:

The visual objective and the safety constraint may require opposite commands, for example when the target is located beyond an obstacle. The arbitration logic must resolve the conflict without producing deadlocks or losing the target permanently.

The situation is concrete: the camera says “the target is ahead, advance”; the LiDAR says “there is a wall ahead”. Both are correct. A naive priority scheme, safety always wins, produces a robot that freezes in front of every obstacle and loses the target. A naive opposite scheme produces a robot that collides.

So how does it decide? Neither command wins outright. The visual controller produces its command first, as if the LiDAR did not exist; then the LiDAR *limits* it, scaling down the forward speed in proportion to how close the nearest obstacle is, and adding a turn towards whichever side has more room. The rotation the camera asked for is never touched.

The result is a robot that slows as it approaches an obstacle, turns away from it while still tracking the target with its camera, and resumes full speed once the way is clear: without either of the two objectives ever being discarded. The next section makes that rule precise.

9.2 How the LiDAR limits the visual command

The rule is simple to state: **the LiDAR never replaces the visual command, it only limits how fast the robot may advance**. Three regimes, depending on the distance d to the nearest obstacle in front (within $\pm 30^\circ$):

Regime	When	What happens
free	$d \geq 0.35$ m	the command passes through unchanged; the LiDAR does not intervene at all
braking	$0.22 < d < 0.35$ m	the forward speed is scaled down linearly, from full at 0.35 m to zero at 0.22; at the same time a rotation towards the freer side is progressively added
stop	$d \leq 0.22$ m	forward motion is cancelled, only the rotation away from the obstacle remains

```
1 def arbitrate(self, v, w):
2     # The LiDAR does not replace the visual command, it limits it:
3     #   beyond safety_distance the command passes through unchanged
4     #   between critical and safety the linear velocity is scaled linearly
5     #   below critical_distance only the turn away from the obstacle remains
6     # Rotation is never suppressed, so the follower can keep the target in
7     # frame while braking: that is what stops the safety layer from causing
8     # the very loss it exists to avoid.
9     if v < 0.0:
10         return self.arbitrate_reverse(v, w)
11
12     d_min, free_side = self.front_obstacle()
13     if d_min >= self.d_safe:
14         return v, w
15     if v == 0.0:
16         # stopped but close to an obstacle: the rotation towards the free
17         # side is added anyway, otherwise standing still in front of a wall
18         # would be a stable state with no way out
19         if d_min <= self.d_crit:
20             return 0.0, self.clamp(w + 0.8 * free_side, self.w_max)
21         return v, w
22     if d_min <= self.d_crit:
23         return 0.0, self.clamp(0.8 * free_side, self.w_max)
24     scale = self.brake_scale(d_min)
25     return v * scale, self.clamp(w + (1.0 - scale) * 0.5 * free_side, self.w_max)
```

Code 3: The whole obstacle avoidance.

Three properties of this rule are worth stating explicitly, because each answers a way the arbitration could have gone wrong.

The braking is gradual, not a switch. Scaling linearly between the two thresholds avoids a discontinuous command, which would appear as a jolt and excite the platform dynamics.

Rotation is never suppressed. Only the forward speed is limited, so the follower can keep turning to hold the target in frame even while braking. This is what prevents the safety layer from *causing* the loss it exists to avoid.

There is always a way out. As braking increases, a rotation towards the freer side is added. Without it, standing still in front of a wall would be a stable state with no exit: the robot would simply stop and stay there. The same term is applied when the commanded speed is already zero, for the same reason.

The rear sector is arbitrated by the same law. The Burger's LiDAR spans a full 360°, so the space behind is known as well as the space ahead; reverse motion is limited to half the forward speed, because opening a gap is never urgent.

10 Experimental Infrastructure

10.1 The target as a reproducible stimulus

The target is not part of the system under test. It is the stimulus, and its required property is *reproducibility*: the comparison between the two strategies is meaningful only if both are exposed to exactly the same scenario, a requirement the proposal states explicitly.

The target is driven in open loop along a fixed waypoint sequence by a dedicated node. It has no sensors and avoids nothing.

10.2 Ground truth for the target, and why it is legitimate

The driver compares the waypoints against the true pose published by the simulator, not against its own odometry. Odometry integrates the wheel encoders, its error accumulates at every turn and is unbounded: after a few laps the target believed it was tens of centimetres from where it actually was, and drove into a wall convinced it had clear space.

This is legitimate, and the distinction matters. The target is not the object of study: it is a scripted actor, the equivalent of a fixed disturbance in a control experiment. What is under test is the *follower*, and the follower perceives the target exclusively through camera and LiDAR. The true pose never enters its control loop, and it is deliberately kept out of the `/tf` tree so that it cannot enter by accident.

11 Testing and Validation

11.1 Experimental protocol

Independent variable. The strategy, and nothing else. Same control law, same gains, same safety arbitration, same trajectory: the two configurations differ in one boolean.

The baseline stops. That is the definition given in the proposal, not an implementation shortcut: “a purely reactive following strategy, which stops as soon as the target is lost”.

Four runs per configuration, 120 seconds each. Repeating the same configuration matters because the simulation is not perfectly repeatable: sensor noise and thread scheduling make two identical runs come out slightly differently. Four runs allow the spread *within* a configuration to be compared against the difference *between* the two, which is what makes a result trustworthy rather than anecdotal.

Two precautions make the runs comparable.

The first concerns how a run is ended. Gazebo does not run in step with the clock on the wall: depending on how busy the machine is, two minutes of simulated time may take four or six minutes to compute. Stopping each run with an ordinary timer would therefore have given the two strategies different amounts of *simulated* time, which is the time that actually matters. Each run is instead stopped by the logging node when the simulation clock has advanced by 120 seconds, so both strategies face exactly the same amount of scenario.

The second is that the two strategies are *alternated* rather than run in blocks, because Gazebo leaves processes behind that degrade the following run; run in blocks, whichever strategy went second would inherit all of that degradation and look worse for a reason unrelated to the strategy itself.

```
./scripts/run_experiments.sh
```

11.2 Metrics

The four indicators committed to in the proposal, plus one breakdown:

Metric	Meaning
tracking_loss_events	times the marker became invisible
definitive_losses	of those, how many escalated to the blind search, the quantity the prediction is expected to reduce
mean_recovery_time_s	mean interval between loss and re-acquisition
rmse_distance_m	how far, on average, the follower is from the distance it should keep. Each sample contributes $d - d^*$; those are squared, averaged and square-rooted: squaring is what stops a metre too far and a metre too close from cancelling out. A value of 0.15 m means the follower typically sits within fifteen centimetres of its setpoint
collisions	samples below the collision threshold

The accuracy is measured against what the camera sees. Not against a reconstructed true position. Three sources of ground-truth position were tried in this simulator and all three proved unusable: the pose topic of a model carries the poses of every entity belonging to it, and the odometry topic turned out to carry two interleaved streams, alternating between points tens of metres apart inside a 10×8 m arena.

The ArUco pose estimate, by contrast, is a real measurement, it is the quantity the control loop acts on, and it is available exactly when the follower is doing its job. It is defined only while the target is visible: with no marker there is no measurement, and inventing one would flatter the strategy that loses the target more often.

11.3 Results

Table 3 reports the campaign in the simple arena, four runs per configuration, 120 s of simulated time each.

Table 3: Reactive versus predictive strategy, simple arena, loop trajectory. Mean $\pm$ standard deviation over four runs.

Metric	Reactive	Predictive	Change
duration_s	120.00 $\pm$ 0.00	120.00 $\pm$ 0.00	–
tracking_loss_events	8.25 $\pm$ 1.71	6.75 $\pm$ 0.96	–18%
definitive_losses	8.00 $\pm$ 1.41	3.75 $\pm$ 0.96	–53%
mean_recovery_time_s	5.87 $\pm$ 3.66	4.35 $\pm$ 0.81	–26%
rmse_distance_m	0.41 $\pm$ 0.33	0.15 $\pm$ 0.01	–63%
collisions	0	0	–

11.3.1 What the comparison shows

The prediction does not reduce how often the target becomes invisible, that is set by the geometry of the route, the same twelve turns every lap. What it changes is what happens next.

Half the losses are resolved before the search is needed. Summed over the four runs of each configuration:

	losses	of which definitive
Reactive	33	32, 97%
Predictive	27	15, 56%

The reactive baseline escalates to the blind search on essentially every loss it suffers. The predictive strategy resolves twelve of its twenty-seven within the three-second horizon, before the recovery behaviour is needed at all.

It keeps the follower at the right distance. The RMS distance error falls from 0.41 m to 0.15. This is the clearest consequence of the mechanism: a follower that stops loses ground while blind and re-acquires the target far from the nominal distance, then spends the rest of the run recovering it. One that keeps advancing re-acquires close to where it should be. The predictive figure is also an order of magnitude more repeatable, ± 0.01 against ± 0.33 .

Recovery time favours the predictive strategy as well, but it is the one figure to read with care: the reactive runs scatter widely, because once the follower has stopped, how long it takes to see the target again depends on where it happened to stop rather than on the strategy. The comparison rests on the definitive losses and the distance error, which are stable across runs.

11.3.2 Why the benefit appears where it does

The prediction does not reduce the number of losses because it cannot: the marker goes edge-on at every turn regardless of what the follower does. What it changes is whether the follower is still close enough to see the target when the panel comes back into view. That is why the improvement shows up in the definitive losses and in the distance error rather than in the loss count.

This also explains why the results are observable at this operating point. The follower works at 1.5 m against a detection range of about 1.8, which leaves little margin. At a more comfortable 1.2 m the margin would be wide enough that the target could not consume it during a single turn, so even a follower that stops would re-acquire it and the two strategies would become indistinguishable to the eye.

11.4 Summary of the work

A complete leader-follower navigation system was developed and evaluated in simulation on the TurtleBot3 platform with ROS2 and Gazebo. The follower tracks a target identified by an ArUco marker using onboard vision alone, holds a prescribed following distance through two decoupled proportional loops with feed-forward of the estimated target velocity, and avoids obstacles through a graded arbitration between the visual objective and the LiDAR constraint.

The distinguishing contribution is the recovery of the target after a loss of visibility, structured in two stages: a bounded blind pursuit on a constant-velocity extrapolation, abandoned either on a fixed horizon or when the LiDAR contradicts the estimate; and, if that fails, a rotation in place until the marker reappears. The design was evaluated by controlled comparison against the purely reactive baseline defined in the proposal, which stops as soon as the target is lost.

The comparison gives a clear answer. Over four runs per configuration, the reactive baseline escalated 32 of its 33 losses of visibility to the blind search, while the predictive strategy resolved 12 of its 27 within the three-second horizon. Definitive losses fall by 53%, the time spent rotating blindly on the spot falls from 37% of the run to 14%, and the RMS distance error falls by 63%, with no collisions in either configuration.

What prediction does *not* do is track better while the target is visible: the control law is identical in both. What it does is keep the follower close enough that a loss stays recoverable.

Deliberately, the system uses no map, no SLAM and no global planner: the navigation is entirely reactive and perception-driven.

11.5 Limitations

The search has a structural ceiling. A genuinely informed search would require a map: knowledge of corridors, rooms and exits would let the follower reason about where a target can possibly have gone. Without one, the search can only rotate in place and wait. This is a consequence of the design decision of Section 1.2 and is accepted as such.

Pose-based following limits the environment. The follower steers at the target's current position rather than along its path, so it cuts corners by an amount that scales with the following distance. It therefore tracks a curve only while the following distance is smaller than the radius

of curvature: at 1.5 m of separation, this excludes any route that wraps an obstacle more tightly than that. The arena used here is dimensioned accordingly.

The motion model is first order. Constant-velocity extrapolation degrades if the target turns while invisible, precisely the case in which it was lost. A model that also estimated the turn rate would in principle do better, but it would have to estimate that rate from the same short, noisy window, and the turn rate is exactly the quantity that becomes unobservable at the moment of the loss. The bounded horizon is what keeps the first-order assumption from mattering: three seconds is short enough that a straight line is still a fair approximation of a route made of straight legs and brief turns.

Odometry drift is uncorrected. Without SLAM or global localisation, the follower's pose estimate in `odom` diverges over time. This is benign over the three-second prediction horizon, which is the only window in which the follower relies on a stored position, and it is the reason the horizon is not longer.

Single target, simulation only. The system tracks one identifier; multi-target tracking would require data association, which is a different problem. All results are obtained in Gazebo: transfer to hardware would require camera calibration and would face harsher lighting, motion blur and a different LiDAR noise model.

11.6 Future work

- **Follow the route, not the target.** The follower steers at where the target is *now*, so it cuts corners: by an amount that grows with the following distance. Steering instead at a point on the route the target has already driven would inherit that route's clearance and remove the limitation. This is the single most valuable extension, and the measurements in this work are what quantify the need for it.
- **Give the search a map.** Without one, a follower that has lost the target can only rotate and wait. Even a rough occupancy grid, built during the pursuit, would let it reason about where a target can plausibly have gone.
- **Estimate the uncertainty, not just the position.** A Kalman filter in place of the sliding-window regression would say *how sure* it is of the estimate, which would allow the prediction horizon to adapt instead of being fixed at three seconds.
- **Drop the marker.** A learned detector would remove the incidence-angle limitation, which is the dominant loss mechanism here, at the cost of a far less certain measurement.

11.7 Repository

The complete source code, the Docker environment, the test suite and the analysis scripts are available at:

<https://github.com/luigipioficco/turtlebot3-leader-follower>